

Clare Oudard

DS210: Gardos

5/2/2025

Exploring Centrality In Road Networks In California: Random vs BFS

A. Project Overview

Goal: Compute and reflect on the centrality measures (especially betweenness and degree) of road networks in CA behave, comparing fully randomized sampling versus randomized BFS sampling

Dataset: roadNet-CA.txt from <https://snap.stanford.edu/data/roadNet-CA.html>

This data set was extremely large and compile time was extremely long due to the fact that there were 1,965,206 nodes and 2,766,607 edges. Used a sample data set with 50,000 edges (larger-sample.txt)

B. Data Processing

I loaded my data into Rust by making a function called `load_graph_from_file` and reading the data set line by line, making a `petgraph:Graph`

Transformed the data by skipping comments and empty lines, mapping every node label that is unique to a `NodeIndex` and adding undirected edges for all node pairs.

C. Code Structure

Modules:

main.rs:

Purpose: Did the sampling, analyzing, and got my outputs in this module

Rationale: Easiest way for me to implement the functions

utils.rs:

Purpose: All of the builder functions I needed for analysis

Rationale: My main function was getting very crowded, and I thought that putting all of my more constructive functions in a different module would be useful.

subgraph.rs:

Purpose: to make a subgraph that is centered around one node and see the difference

Rationale: so that it is easier to isolate and update the subgraphs

test.rs:

Purpose: to test the key functions used to make sure program runs smoothly

Rationale: I prefer to keep everything separate and the tests were crowding the space in utils.rs

Key Functions:

1. **load_graph_from_file**

Purpose: Loads edge list from file into a `petgraph::Graph`.

Inputs: &str path to a file where each line represents an edge: node1 node2.

Outputs: A `Graph<(), (), petgraph::Undirected>` as the graph

Core Logic: Iterates through lines of the file, adds nodes and edges to a petgraph graph by using a HashMap.

2. **normalize_scores**

Purpose: Normalizes a centrality score HashMap so values fall between 0 and 1.

Inputs: `HashMap<NodeIndex, f64>` with raw scores.

Outputs: `HashMap<NodeIndex, f64>` with normalized scores.

Core Logic: Finds the maximum value and divides all scores by it (if max > 0).

3. **rank_top_n**

Purpose: Finds the top-N nodes based on a centrality measure.

Inputs: `HashMap<NodeIndex, f64>` and integer n.

Outputs: Sorted `Vec<(NodeIndex, f64)>` of the top n nodes.

Core Logic: Collects scores into vector, sorts by descending value, takes top n.

4. **degree_centrality**

Purpose: Computes normalized degree centrality.

Inputs: An `UndirectedGraph`.

Outputs: `HashMap<NodeIndex, f64>`

Core Logic: For each node, computes degree and divides by n - 1 for normalization.

5. **bfs_sample_nodes**

Purpose: Samples fixed number of nodes by performing a BFS from a root.

Inputs: A graph, a starting node, and a node limit.

Outputs: `Vec<NodeIndex>` of sampled nodes.

Core Logic: BFS traversal using queue and visited set, stops once the sample size is reached.

6. **betweenness centrality sampled**

Purpose: Estimates betweenness centrality by finding shortest paths from a subset of nodes.

Inputs: Graph and `&[NodeIndex]` sampled source nodes.

Outputs: `HashMap<NodeIndex, f64>` with betweenness values.

Core Logic: For each source, shows shortest paths using BFS predecessors and counts how often nodes appear in these paths (not including endpoints).

7. **extract_neighborhood_subgraph**

Purpose: Constructs subgraph of nodes that are within k hops of a node.

Inputs: A graph, a center `NodeIndex`, and integer radius k.

Outputs: A new `Graph<(), (), Undirected>` containing only the neighborhood.

Core Logic: BFS from center, adding every node or edge up to distance k.

Main Workflow:

1. Load the graph using `load_graph_from_file` in `utils.rs`
2. Sample the nodes: (two methods)
 - a. Random: picks 50 nodes at random
 - b. BFS: calls `bfs_sample_nodes` from `utils.rs` to get local sample
3. Calculate betweenness centrality: both random and bfs passed into `betweenness centrality sampled` from `utils.rs` to estimate centrality
4. Normalize results: using `normalize_scores` in `utils.rs`
5. Analyze the Subgraph: most central node is selected and `extract_neighborhood_subgraph` from `subgraph.rs` builds local subgraph around that node and centrality is repeated on subgraph
6. Print Results: using `main()` function

D. Tests

cargo test outputs:

running 5 tests

test test::test_betweenness centrality sampled ... ok

test test::test_bfs_sample_nodes ... ok

test test::test_degree centrality simple ... ok

test test::test_rank_top_n ... ok

test test::test_normalize_scores_range ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Every test, what it checks, why it matters:

create_test_graph:

What it does: makes a small undirected graph with four nodes and three edges for all of the other tests

Why it matters: Makes it so that it is easy for me to assert the results I should be getting

test_degree centrality_simple:

What it checks: that degree centrality is correct for a tiny, simple graph

Why it matters: if my degree centrality is off for a small graph, then it will be very off for as large a graph as I am working with in my data set

test_normalize_scores_range:

What it checks: that normalize_scores is returning values in the range [0, 1]

Why it matters: if my normalization is skewed, the results will be meaningless and any interpretations of the data will be useless.

test_rank_top_n:

What it checks: makes sure that the top n highest scoring nodes are being returned in descending order

Why it matters: makes sure that my hierarchy is correct so that the ranking logic will be correct in use

test_bfs_sample_nodes:

What it checks: that BFS sampling expands correctly and doesn't go past the limit I set for it

Why it matters: makes sure that I am actually sampling within the neighborhood of the node and makes sure that the BFS can be reproduced

test_betweenness centrality_sampled:

What it checks: that any central node is not being given a zero betweenness score

Why it matters: because the betweenness is essential for my results, if my betweenness scores are not working correctly then all my results are meaningless

E. Results

Random sampling:

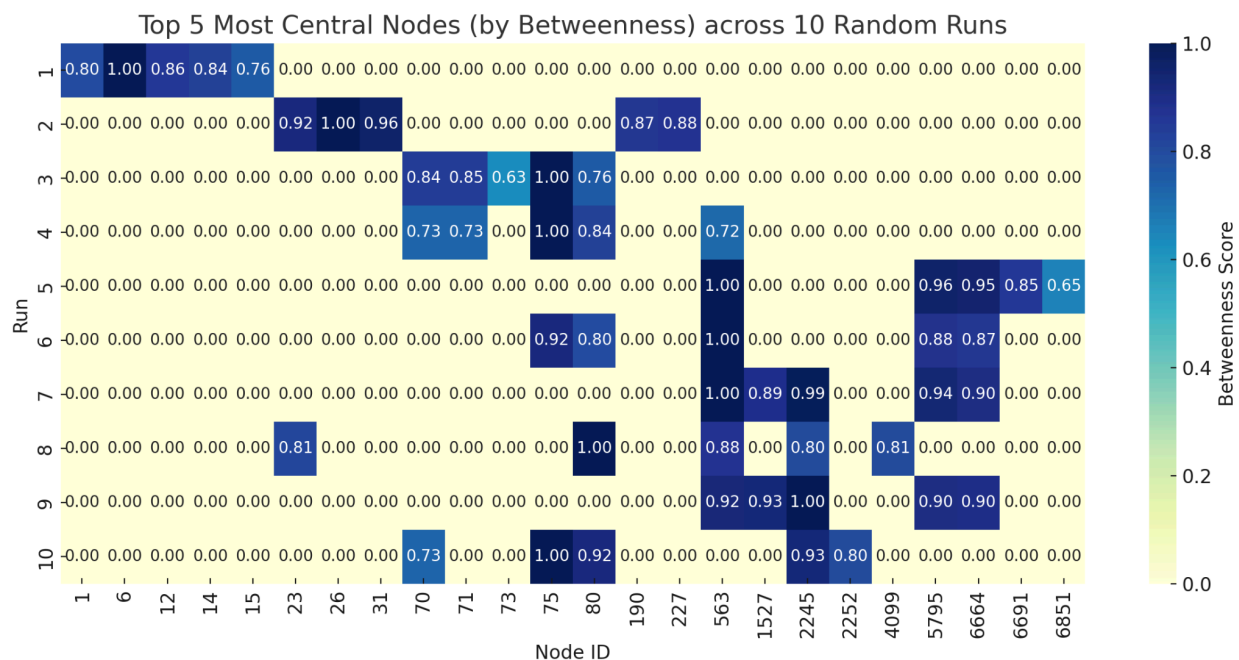
Console Output:

Top 5 Most Central Nodes (by Betweenness):

Node 6664: 1.000000
Node 5795: 0.990414
Node 6691: 0.985027
Node 9959: 0.979306
Node 563: 0.752567

Interpretation:

Because of randomness, it is important to include the data I collected from running this many times. I plugged my raw data into ChatGPT and it made this helpful heatmap of the most central nodes. The heatmap reveals a weak negative correlation between node rank and frequency of appearance in the top 5, suggesting a distribution where only a few nodes dominate.



Observations:

Most nodes have zero or close to zero degree, indicating low betweenness.

A few nodes show high betweenness despite zero degree → degree alone doesn't determine betweenness

Betweenness reveals more about connectivity across paths.

Node 9959 consistently appears at or near the top

Nodes 6664, 5795, and 6691 also appear repeatedly, with high but slightly variable scores.

Randomized BFS Sampling

Console Output:

Starting BFS at node: 17397

Top 5 Most Central Nodes (by Betweenness via Randomized BFS):

Node 16095: 1.000000

Node 16106: 0.901021

Node 16105: 0.886717

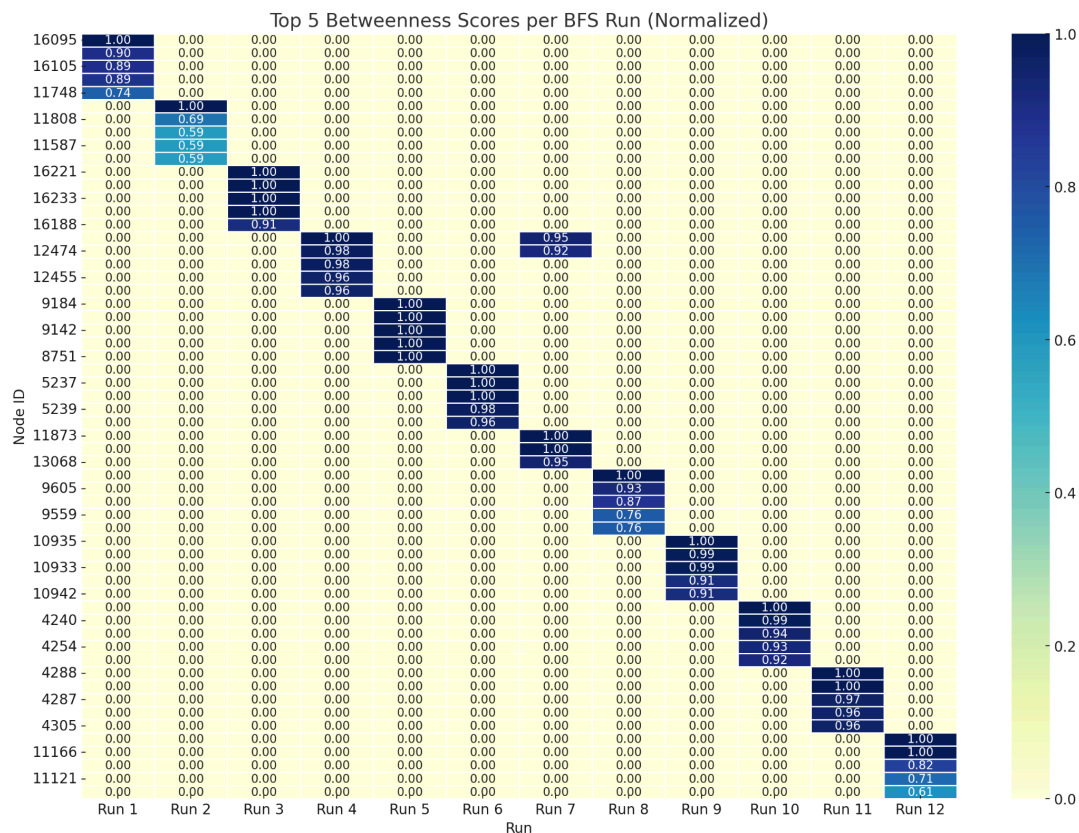
Node 16419: 0.885711

Node 11748: 0.736826

Interpretation:

I then did randomized BFS sampling (randomly choose a starting node and perform BFS on it)

I ran it over ten times, plugged my results into ChatGPT and got this heatmap, displaying a very strong negative linear trend, showing that nodes that dominate are rarer in BFS sampling.



Observations:

Centrality is very clustered (as a result of the BFS)

Nodes that consistently appear in the top 5 (e.g., 12483, 11776, 4288)
Showing central regions

Comparing BFS vs Random Sampling:

Console Output:

Starting at node: 12045

Top 5 Most Central Nodes (BFS Sample from 12045):

Node 12071: 1.000000

Node 9657: 0.947137

Node 9562: 0.932095

Node 9605: 0.825281

Node 9073: 0.820129

Top 5 Most Central Nodes (Random Sample):

Node 6664: 1.000000

Node 5795: 0.994884

Node 6691: 0.981454

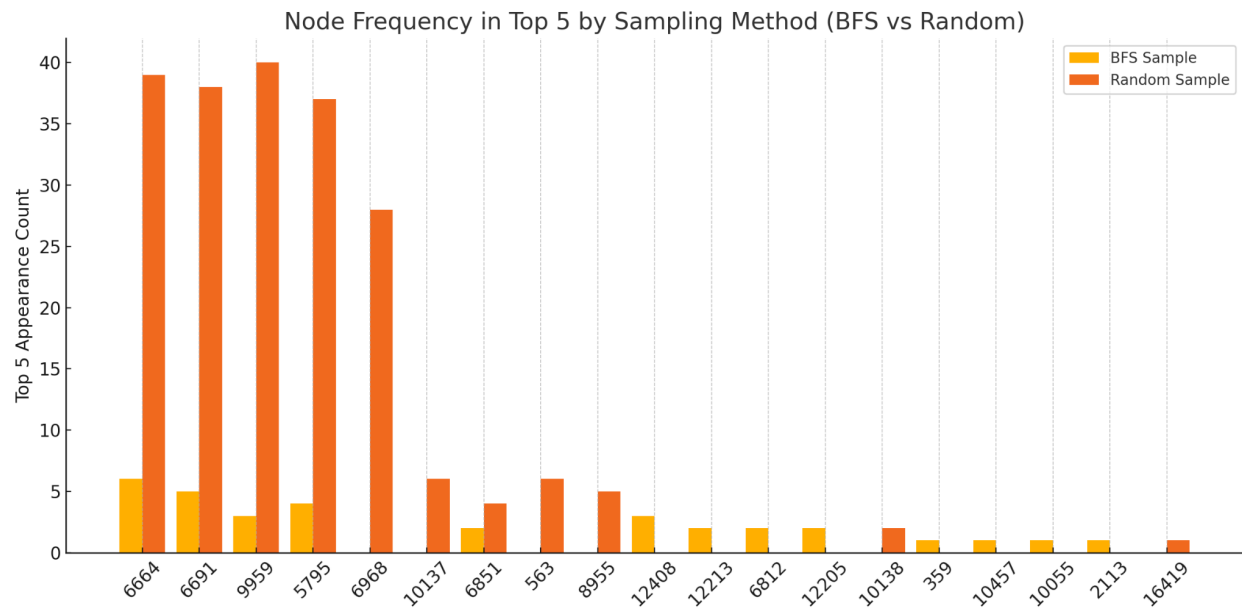
Node 9959: 0.979263

Node 563: 0.813636

Starting at node: 9942

Interpretation:

I then made it so that my code would calculate both at the same time, making it easier to calculate the two at the same time and giving meaningful results to people viewing my code. I plugged my results of about 40 runs into ChatGPT and asked it to make a graph comparing the two.



From the graph we can see that random sampling finds the highest centrality nodes while BFS has a wider variety of nodes that it identifies, which is aligned with what the heat map showed us.

Takeaways:

Random Sampling:

Pros: finds globally central nodes

Cons: will test isolated nodes that are not central and do not provide meaningful results

Randomized BFS Sampling:

Pros: finds locally central nodes

Cons: is dependent on starting point, so not as good for global centrality

Which one is better in this case?

I think that for road networks for all of California, locally important nodes that might not be globally dominant but are critical within their region can be more relevant and helpful than the globally important nodes.

F. Usage Instructions:

Use larger-sample.txt rather than full data set (made by doing `head -n 50000 roadNet-CA.txt > larger-sample.txt`)

Run `cargo run --release`

Will output results

With that specific sample, takes around 15 seconds to finish running

G. AI Disclosure:

Every graph and visual aid was made by ChatGPT, based on raw output from my program. Because they are all graphs, I think it would be trivial to explain for every one, but overall the graphs gave me a deeper understanding of my data as it helpfully made it so that I could see trends and the relationships between the two sampling styles.

Here are sum of my results, would you mind putting it into a graph or table: Format for output:
Closeness Centrality (sampled):

Node 0: 0.288889
Node 1: 0.295455
Node 2: 0.317073
Node 3: 0.247619
Node 4: 0.305882
Node 5: 0.238532
Node 6: 0.361111
Node 7: 0.200000
Node 8: 0.268041
Node 9: 0.342105

Normalized Betweenness Centrality (Sampled):

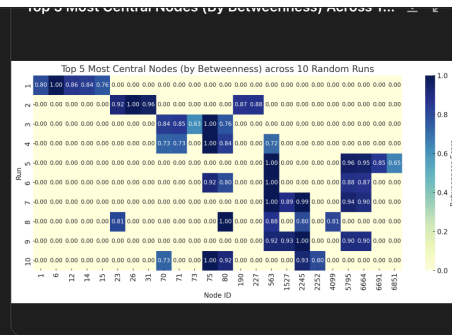
Node 9: 0.468354
Node 1: 0.797468
Node 4: 0.405063
Node 14: 0.835443
Node 13: 0.278481
Node 15: 0.75[^]194
Node 16: 0.17, -15

Here's a heatmap showing the top 5 most central nodes (by normalized betweenness centrality) across 10 different random samples. You can clearly see:

- Node 563 is dominant — appearing as the top node in most runs.
- Nodes 2245, 75, 80, 5795, and 6664 also appear frequently, indicating they are consistently central in different slices of the network.
- Some nodes like 1527, 4099, and 2252 are more intermittent, suggesting context-dependent importance.

This visually supports your insight: some nodes are globally central, while others are only central within certain substructures of the network.

Would you like to generate a second plot showing subgraph-level centrality patterns as well? [-]



Here's a heatmap showing the top 5 most central nodes (by normalized betweenness centrality) across 10 different random samples. You can clearly see:

- Node 563 is dominant — appearing as the top node in most runs.
- Nodes 2245, 75, 80, 5795, and 6664 also appear frequently, indicating they are consistently central in different slices of the network.
- Some nodes like 1527, 4099, and 2252 are more intermittent, suggesting context-

Ask anything



Search



Deep research



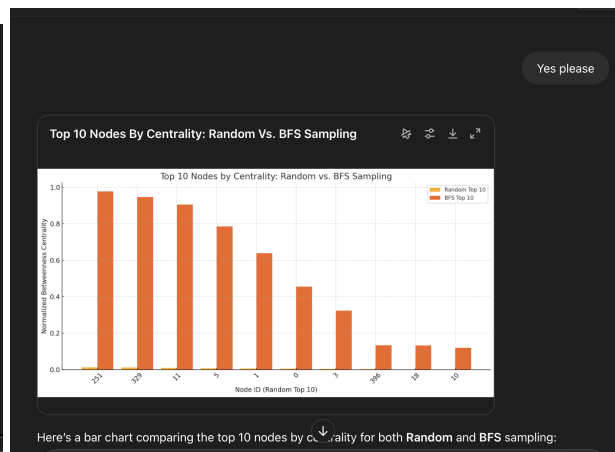
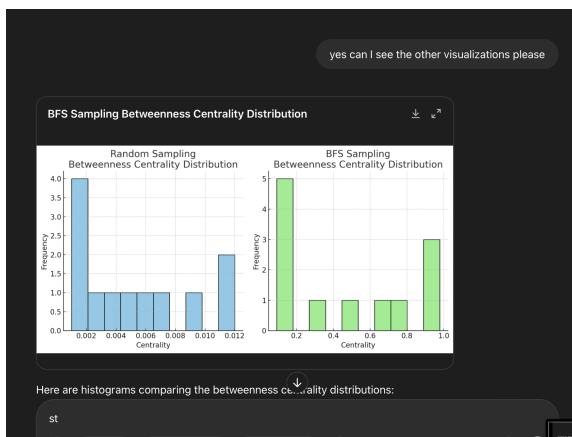
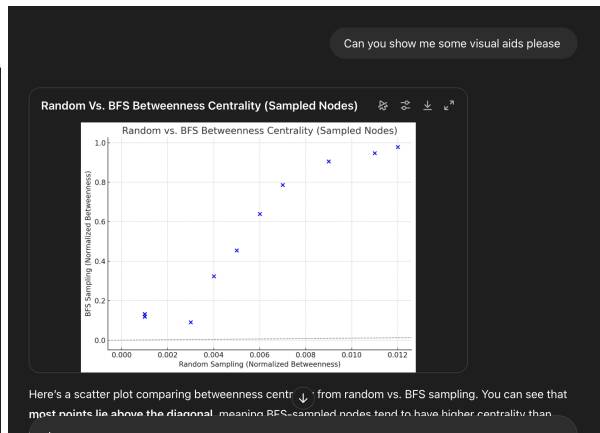
Create image



Here are the results from my comparisons: Node Random BFS
5250 0.001 0.009
308 0.000 0.021
398 0.000 0.000
13934 0.002 0.000
10645 0.000 0.000
5 0.004 0.786
883 0.000 0.000
251 0.007 0.978
7 0.002 0.248

5206 0.000 0.000
 12797 0.000 0.000
 14181 0.000 0.000
 727 0.000 0.000
 724 0.001 0.024
 422 0.006 0.064
 3 0.026 0.324
 318 0.000 0.053
 303 0.000 0.018
 397 0.000 0.000
 13 0.000 0.048
 4 0.002 0.148
 6846 0.121 0.000
 2744 0.001 0.000
 294 0.003 0.016
 8509 0.042 0.000
 1849 0.000 0.000
 11721 0.000 0.000
 14440 0.000 0.000

Thanks for sharing your full results — that's a strong and insightful dataset! Here's what I recommend:

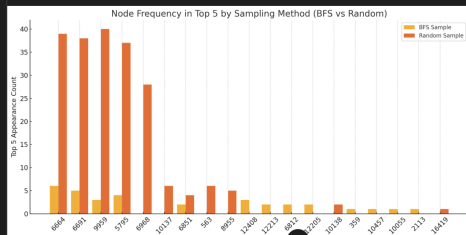


Node 9959: 0.937444

Node 563: 0.836099

If you could provide another visual aid and maybe one that makes the comparison between the two very clear

Node Frequency In Top 5 By Sampling Method (BFS Vs Ran...



Can you make me some graphs that interpret these results? Additionally, should I run the code some more for more meaningful results, or is this sufficient? Starting BFS at node: 17397

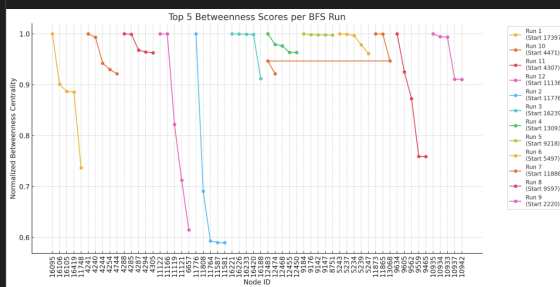
Top 5 Most Central Nodes (by Betweenness via Randomized BFS):
Node 16095: 1.000000
Node 16106: 0.901021
Node 16105: 0.886717
Node 16419: 0.885711
Node 11748: 0.736826

Starting BFS at node: 11776

Top 5 Most Central Nodes (by Betweenness via Randomized BFS):
Node 11776: 1.000000
Node 11808: 0.690651
Node 11764: 0.592831
Node 11587: 0.589897
Node 11581: 0.589287

=== RUN 1 ===

Top 5 Betweenness Scores Per BFS Run



Here is a line plot showing the top 5 nodes with the highest normalized betweenness centrality for each BFS run. Each line represents one run, labeled by the starting node.

Should you run it more?

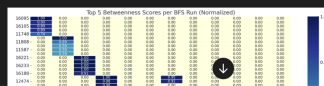
You've already run it over 10 times, which gives a solid base for qualitative comparison. If you're writing a report or seeking stronger evidence:

- A few more runs (e.g., 3–5) can help reinforce patterns.
- Consider computing the **average betweenness** across runs per node if overlaps exist.

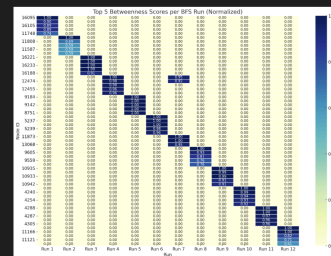
Would you like a second plot (like a heatmap or bar graph) summarizing node frequency or average centrality? [~]

I think a heat map would be helpful

Top 5 Betweenness Scores Per BFS Run (Normalized)



Top 5 Betweenness Scores Per BFS Run (Normalized)



Here's a heatmap showing the top 5 betweenness centrality scores across multiple BFS runs. This visualization helps you quickly spot:

- Nodes that consistently appear in the top 5 (e.g., 12483, 11776, 4288)